```react import React, { useState, useEffect, useRef } from 'react'; import { Server, Cpu, HardDrive, Terminal, MessageSquare, Play, Square, RefreshCw, Edit3, Plus, X, Send, ShieldAlert, Zap, Activity, Box, Sliders, ChevronRight, Copy, Check, Search, Filter, Layers, Database, Globe, Radio, Settings, AlertCircle } from 'lucide-react'; // --- INITIAL MOCK DATA --- const INITIAL_AGENTS = [{ id: 'agent-101', name: 'Customer Support Bot', image: 'cagent/support-v2:latest', status: 'Running', // Running, Stopped, Deploying model: 'gpt-4o-mini', provider: 'OpenAI', uptime: '14d 6h 22m', cpu: '1.2%', memory: '142 MB', tools: ['Zendesk MCP', 'PostgreSQL Docs', 'Email Sender'], threadsCount: 24, yaml: `version: "1.0" agent: name: "Customer Support Bot" version: "2.1.0" model: provider: "openai" name: "gpt-4o-mini" temperature: 0.3 system_instructions: | You are an autonomous customer support agent for enterprise clients. Always remain polite, concise, and pull relevant docs before answering. mcp_servers: - name: zendesk-mcp url: "http://zendesk-mcp:8080/mcp" - name: db-mcp url: "http://pg-mcp:5432/mcp" capabilities: - search_knowledge_base - escalate_ticket - send_reply_email` }, { id: 'agent-102', name: 'SQL Analytics Specialist', image: 'cagent/db-analyst:v1.4', status: 'Running', model: 'claude-3-5-sonnet', provider: 'Anthropic', uptime: '2d 11h 05m', cpu: '8.4%', memory: '410 MB', tools: ['BigQuery Connector', 'Python REPL', 'Slack Bot'], threadsCount: 8, yaml: `version: "1.0" agent: name: "SQL Analytics Specialist" version: "1.4.0" model: provider: "anthropic" name: "claude-3-5-sonnet" temperature: 0.1 system_instructions: | You convert natural language analytics requests into optimized SQL. Always explain query performance implications before running updates. mcp_servers: - name: bigquery-mcp url: "http://bq-mcp:9000/mcp" capabilities: - run_read_only_queries - output_chart_json` }, { id: 'agent-103', name: 'DevOps Alert Triage', image: 'cagent/devops-sentinel:latest', status: 'Stopped', model: 'gemini-1.5-pro', provider: 'Google AI', uptime: '0m', cpu: '0.0%', memory: '0 MB', tools: ['Kubernetes API', 'Datadog MCP', 'PagerDuty'], threadsCount: 0, yaml: `version: "1.0" agent: name: "DevOps Alert Triage" version: "1.0.0" model: provider: "google" name: "gemini-1.5-pro" temperature: 0.2 system_instructions: | Monitors incoming PagerDuty alerts and queries k8s state to provide instant diagnosis. mcp_servers: - name: k8s-gateway url: "http://k8s-mcp:3000" capabilities: - fetch_pod_logs - restart_deployment` }, { id: 'agent-104', name: 'Market Research Miner', image: 'cagent/researcher:v3.0', status: 'Running', model: 'deepseek-r1', provider: 'DeepSeek Local', uptime: '5h 42m', cpu: '24.1%', memory: '1.2 GB', tools: ['Brave Search MCP', 'PDF Parser', 'Notion Writer'], threadsCount: 3, yaml: `version: "1.0" agent: name: "Market Research Miner" version: "3.0.0" model: provider: "ollama" name: "deepseek-r1" temperature: 0.6 system_instructions: | Synthesize research on technical markets and compile markdown digests. mcp_servers: - name: web-search url: "http://brave-mcp:8000" capabilities: - scrape_web - extract_tables` }]; const INITIAL_THREADS = [{ id: 'th-8831', agentId: 'agent-101', agentName: 'Customer Support Bot', user: 'enterprise_user_99@acme.com', status: 'active', lastActive: 'Just now', messagesCount: 14, messages: [{ id: 1, sender: 'user', text: 'Hi, our API rate limit is throwing 429 errors suddenly. Can you check our quota?', timestamp: '14:20:11' }, { id: 2, sender: 'agent', text: 'I am looking into your account tier and current rate logs using the Postgres MCP database tool...', timestamp: '14:20:13', toolCall: 'db-mcp.query_usage({ user_id: "usr_99" })' }, { id: 3, sender: 'agent', text: 'I see that user `usr_99` hit the 10,000 req/min limit on Tier-2 at 14:18 UTC. Would you like me to issue a temporary 24-hour quota bump to Tier-3?', timestamp: '14:20:15' }, { id: 4, sender: 'user', text: 'Yes please, we are running a migration batch!', timestamp: '14:21:02' }] }, { id: 'th-8832', agentId: 'agent-102', agentName: 'SQL Analytics Specialist', user: 'data_lead@internal.io', status: 'idle', lastActive: '12m ago', messagesCount: 6, messages: [{ id: 1, sender: 'user', text: 'Generate a query showing top 5 churned customers this quarter.', timestamp: '14:05:00' }, { id: 2, sender: 'agent', text: 'Here is the recommended query utilizing our BigQuery warehouse views:', timestamp: '14:05:04', code: `SELECT customer_id, sum(arr_lost) as total_arr\nFROM analytics.churn_events\nWHERE event_date >= '2026-07-01'\nGROUP BY 1 ORDER BY 2 DESC
```

```
LIMIT 5;` } ] }, { id: 'th-8833', agentId: 'agent-101', agentName: 'Customer Support Bot', user: 'client_bob@megacorp.com', status: 'active', lastActive: '2m ago', messagesCount: 22, messages: [ { id: 1, sender: 'user', text: 'How do I enable SSO with Okta in the dashboard?', timestamp: '14:18:20' }, { id: 2, sender: 'agent', text: 'To configure Okta SSO, navigate to Settings > Security > SAML 2.0 configuration and upload your Identity Provider Metadata XML file.', timestamp: '14:18:22' } ] } ]; const INITIAL_LOGS = [ '[SYS] [INFO] Container runtime initializing OCI engine...', '[AGENT-101] [INFO] MCP Server connection established with zendesk-mcp:8080', '[AGENT-101] [INFO] Registered tool capabilities: [search_knowledge_base, escalate_ticket, send_reply_email]', '[AGENT-102] [METRIC] Memory snapshot: 410MB / 1024MB allocated', '[AGENT-104] [INFO] DeepSeek Local host responded in 112ms (status: healthy)', '[SYS] [WARN] Rate limit threshold warning on Google AI API gateway (82% capacity)', '[AGENT-101] [EXEC] Invoked tool zendesk-mcp.query_usage for thread th-8831', '[SYS] [INFO] Heartbeat check passed for 3 active agent containers.' ]; export default function App() { // Navigation State const [activeTab, setActiveTab] = useState('agents'); // agents, threads, logs, settings // Core Data States const [agents, setAgents] = useState(INITIAL_AGENTS); const [threads, setThreads] = useState(INITIAL_THREADS); const [logs, setLogs] = useState(INITIAL_LOGS); // Filter States const [agentFilter, setAgentFilter] = useState(''); const [selectedAgentId, setSelectedAgentId] = useState(null); // Thread Active State const [selectedThreadId, setSelectedThreadId] = useState('th-8831'); const [chatInput, setChatInput] = useState(''); const [overrideInput, setOverrideInput] = useState(''); // Modals & Editors const [editingYamlAgent, setEditingYamlAgent] = useState(null); const [yamlContent, setYamlContent] = useState(''); const [isDeployModalOpen, setIsDeployModalOpen] = useState(false); const [newAgentYaml, setNewAgentYaml] = useState(`version: "1.0" agent: name: "New Enterprise Assistant" version: "1.0.0" model: provider: "openai" name: "gpt-4o" temperature: 0.4 system_instructions: | Enter your automated agent persona and instructions here... mcp_servers: - name: web-search url: "http://search-mcp:8000" capabilities: - browse_web - output_markdown`); // Telemetry state simulation const [metrics, setMetrics] = useState({ cpu: 12.8, ram: 1.8, activeContainers: 3, activeApiConns: 18 }); const chatEndRef = useRef(null); const logEndRef = useRef(null); // Simulated live metric stream & real-time log ingestion useEffect(() => { const interval = setInterval(() => { setMetrics(prev => ({ cpu: Math.min(95, Math.max(5, (prev.cpu + (Math.random() * 4 - 2)).toFixed(1))), ram: Math.min(8, Math.max(0.5, (prev.ram + (Math.random() * 0.1 - 0.05)).toFixed(2))), activeContainers: agents.filter(a => a.status === 'Running').length, activeApiConns: Math.floor(15 + Math.random() * 8) })); // Random synthetic log event if (Math.random() > 0.6) { const randAgent = agents[Math.floor(Math.random() * agents.length)]; const events = [ `[${randAgent.id.toUpperCase()}] [DEBUG] Model inference request completed in ${(Math.random() * 400 + 100).toFixed(0)}ms`, `[SYS] [HEALTHCHECK] Container ${randAgent.image} ping: OK`, `[${randAgent.id.toUpperCase()}] [MCP] Context window refreshed (${Math.floor(Math.random() * 2000 + 1000)} tokens)` ]; const newLog = events[Math.floor(Math.random() * events.length)]; setLogs(prev => [...prev.slice(-40), newLog]); } }, 3000); return () => clearInterval(interval); }, [agents]); // Auto scroll chat / logs useEffect(() => { chatEndRef.current?.scrollIntoView({ behavior: 'smooth' }); }, [threads, selectedThreadId]); useEffect(() => { logEndRef.current?.scrollIntoView({ behavior: 'smooth' }); }, [logs]); // Helper Actions const toggleAgentStatus = (id) => { setAgents(agents.map(a => { if (a.id === id) { const newStatus = a.status === 'Running' ? 'Stopped' : 'Running'; const newLog = `[SYS] [ACTION] User switched container ${a.id} (${a.name}) status to: ${newStatus}`; setLogs(prev => [...prev, newLog]); return { ...a, status: newStatus, cpu: newStatus === 'Stopped' ? '0.0%' : `${(Math.random() * 10).toFixed(1)}%`, memory: newStatus === 'Stopped' ? '0 MB' : '150 MB' }; } return a; })); }; const handleOpenYamlEditor = (agent) => { setEditingYamlAgent(agent); setYamlContent(agent.yaml); }; const handleSaveYaml = () => { if (!editingYamlAgent) return; setAgents(agents.map(a => a.id === editingYamlAgent.id ? { ...a, yaml: yamlContent } : a));
```

setLogs(prev => [...prev, `[SYS] [CONFIG] Re-deployed YAML config for agent ${editingYamlAgent.id}`]); setEditingYamlAgent(null); }; const handleDeployNewAgent = () => { try { // Basic extraction from YAML string for demo const nameMatch = newAgentYaml.match(/name:\s*"([^"]+)"/); const name = nameMatch ? nameMatch[1] : 'Custom Agent Container'; const providerMatch = newAgentYaml.match(/provider:\s*"([^"]+)"/); const provider = providerMatch ? providerMatch[1].toUpperCase() : 'Custom API'; const newAgent = { id: `agent-${Math.floor(100 + Math.random() * 900)}`, name: name, image: `cagent/${name.toLowerCase().replace(/\s+/g, '-')}:v1.0`, status: 'Running', model: 'custom-mcp-model', provider: provider, uptime: 'Just deployed', cpu: '2.1%', memory: '180 MB', tools: ['Custom MCP Gateway'], threadsCount: 0, yaml: newAgentYaml }; setAgents([newAgent, ...agents]); setLogs(prev => [...prev, `[SYS] [DEPLOY] Container image cagent/${newAgent.id}:v1.0 successfully spun up!`]); setIsDeployModalOpen(false); } catch (e) { alert("Error parsing YAML config format."); } }; const handleSendMessage = (e) => { e.preventDefault(); if (!chatInput.trim()) return; const currentThread = threads.find(t => t.id === selectedThreadId); if (!currentThread) return; const userMsg = { id: Date.now(), sender: 'user', text: chatInput, timestamp: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second: '2-digit' }) }; const updatedThreads = threads.map(t => { if (t.id === selectedThreadId) { return { ...t, lastActive: 'Just now', messages: [...t.messages, userMsg] }; } return t; }); setThreads(updatedThreads); setChatInput(''); // Simulate Agent automated reply after brief lag setTimeout(() => { const agentReply = { id: Date.now() + 1, sender: 'agent', text: `[Container Response] Processed input through runtime context. Request received and executing instructions.`, timestamp: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second: '2-digit' }) }; setThreads(prev => prev.map(t => { if (t.id === selectedThreadId) { return { ...t, messages: [...t.messages, agentReply] }; } return t; })); }, 1200); }; const handleSendPromptOverride = () => { if (!overrideInput.trim()) return; const sysMsg = { id: Date.now(), sender: 'system', text: `⚡ PROMPT OVERRIDE INJECTED: "${overrideInput}"`, timestamp: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }) }; setThreads(threads.map(t => t.id === selectedThreadId ? { ...t, messages: [...t.messages, sysMsg] } : t)); setLogs(prev => [...prev, `[ADMIN OVERRIDE] Thread ${selectedThreadId} received system prompt intervention: ${overrideInput}`]); setOverrideInput(''); }; const selectedThread = threads.find(t => t.id === selectedThreadId) || threads[0]; const activeAgentForThread = agents.find(a => a.id === selectedThread?.agentId); const filteredAgents = agents.filter(a => a.name.toLowerCase().includes(agentFilter.toLowerCase()) || a.image.toLowerCase().includes(agentFilter.toLowerCase()) || a.provider.toLowerCase().includes(agentFilter.toLowerCase())); return ({/* --- SIDEBAR NAVIGATION --- */} {/* Header Branding */}

CAGENT ENGINE

v2.4.0 Container Core

{/* Navigation Links */}

setActiveTab('agents')} className={`w-full flex items-center justify-between px-3 py-2.5 rounded-lg text-sm font-medium transition-all ${ activeTab === 'agents' ? 'bg-indigo-600/20 text-indigo-300 border border-indigo-500/30' : 'text-slate-400 hover:bg-slate-800/60 hover:text-slate-200' }`} >
Agents & Containers
{agents.length}

setActiveTab('threads')} className={`w-full flex items-center justify-between px-3 py-2.5 rounded-lg text-sm font-medium transition-all ${ activeTab === 'threads' ? 'bg-indigo-600/20 text-indigo-300 border border-indigo-500/30' : 'text-slate-400 hover:bg-slate-800/60 hover:text-slate-200' }`} >
Thread Manager
{threads.length} live

setActiveTab('logs')} className={`w-full flex items-center justify-between px-3 py-2.5 rounded-lg text-sm font-medium transition-all ${ activeTab === 'logs' ? 'bg-indigo-600/20 text-indigo-300 border border-indigo-500/30' : 'text-slate-400 hover:bg-slate-800/60 hover:text-slate-200' }`} >
Logs & Telemetry

setActiveTab('settings')} className={`w-full flex items-center justify-between px-3 py-2.5 rounded-lg text-sm font-medium transition-all ${ activeTab === 'settings' ? 'bg-indigo-600/20 text-indigo-300 border border-indigo-500/30' : 'text-slate-400 hover:bg-slate-800/60 hover:text-slate-200' }`} >
MCP Gateways

{/* System Node Telemetry Snippet */}
Node: local-docker READY
CPU Load {metrics.cpu}%
Memory {metrics.ram} GB
{/* --- MAIN CONTENT AREA --- */}
{/* TOP STATUS TELEMETRY BAR */}
ACTIVE CONTAINERS: {metrics.activeContainers}
API CONNS: {metrics.activeApiConns}
MCP PROTOCOL: v2024.11.05

setIsDeployModalOpen(true)} className="flex items-center space-x-2 bg-gradient-to-r from-indigo-600 to-cyan-600 hover:from-indigo-500 hover:to-cyan-500 text-white px-4 py-2 rounded-lg font-medium text-sm transition-all shadow-lg shadow-indigo-600/20 active:scale-95" > Deploy New Agent Package

{/* TABS CONTAINER BODY */}
{/* TAB 1: AGENTS & CONTAINERS DASHBOARD */} {activeTab === 'agents' && (
{/* Search & Filter Bar */}
{agentFilter} setAgentFilter(e.target.value)} className="w-full bg-slate-950 border border-slate-800 rounded-lg pl-9 pr-4 py-2 text-sm text-slate-200 placeholder-slate-500 focus:outline-none focus:border-indigo-500/50" />
Total Deployed: {filteredAgents.length}
{/* Agent Cards Grid */}
{filteredAgents.map(agent => (
{/* Card Header */}

{agent.name}

● {agent.status}

{agent.image}

{/* Power Control */}

toggleAgentStatus(agent.id)} title={agent.status === 'Running' ? 'Stop Container' : 'Start Container'} className={`p-2 rounded-lg border transition-all ${ agent.status === 'Running' ? 'bg-rose-500/10 border-rose-500/30 text-rose-400 hover:bg-rose-500/20' : 'bg-emerald-500/10 border-emerald-500/30 text-emerald-400 hover:bg-emerald-500/20' }`} > {agent.status === 'Running' ? : }

{/* Card Body & Specs */}
{/* Key-Value Specs */}
Model Provider {agent.provider}
Model Engine {agent.model}
Uptime {agent.uptime}
{/* Hardware footprint */}
CPU: {agent.cpu} RAM: {agent.memory} Active Threads: {agent.threadsCount}
{/* Configured MCP Capabilities */}
CONNECTED TOOLS & MCP SERVERS:
{agent.tools.map((tool, idx) => ({tool}))}
{/* Card Actions Footer */}

handleOpenYamlEditor(agent)} className="flex items-center space-x-1.5 text-slate-400 hover:text-indigo-400 transition-colors text-xs font-mono" > Edit Config (YAML)

{ setSelectedThreadId(threads.find(t => t.agentId === agent.id)?.id || threads[0].id); setActiveTab('threads'); }} className="flex items-center space-x-1 text-cyan-400 hover:text-cyan-300 text-xs font-medium" > View Threads

))}
)} {/* TAB 2: ACTIVE CONVERSATION & THREAD MANAGER */} {activeTab === 'threads' && (
{/* Sidebar List of Threads */}

Active User Threads

Live customer execution sessions

{threads.map(t => (

setSelectedThreadId(t.id)} className={`w-full p-3.5 text-left transition-all flex flex-col gap-1 ${ selectedThreadId === t.id ? 'bg-indigo-600/15 border-l-4 border-indigo-500 text-slate-100' : 'hover:bg-slate-800/40 text-slate-400' }`} >
{t.id} {t.lastActive}
{t.user}
{t.agentName} {t.messages.length} msgs

))}
{/* Main Live Interactive Chat Stream */}
{/* Thread Header */}

{selectedThread?.id}

Customer: {selectedThread?.user}

Agent: {selectedThread?.agentName}
{/* Messages Display Scroll */}
{selectedThread?.messages.map((msg) => (
{/* System Injected Prompt Badge */} {msg.sender === 'system' ? (
{msg.text}
) : (
{msg.sender === 'user' ? 'CUSTOMER' : activeAgentForThread?.name || 'AGENT'}
{msg.timestamp}

{msg.text}

{/* MCP Tool Trigger Snippet if present */} {msg.toolCall && (
MCP Tool Invocated

{msg.toolCall}
)} {/* Code Block if present */} {msg.code && (

{msg.code}

)}
)}
)))}
{/* Prompt Intervention & Direct Input Controls */}
{/* Admin Direct Intervention Prompt Overlay */}
{overrideInput} setOverrideInput(e.target.value)} className="flex-1 bg-slate-950 border border-amber-500/30 rounded-md px-3 py-1.5 text-xs text-slate-200 placeholder-slate-500 focus:outline-none focus:border-amber-500/60 font-mono" /> Override
{/* Standard Message Input Bar */}
{chatInput} setChatInput(e.target.value)} className="flex-1 bg-slate-950 border border-slate-800 rounded-lg px-4 py-2 text-sm text-slate-200 placeholder-slate-500 focus:outline-none focus:border-indigo-500" />
{/* System Prompt Context Window Inspector Pane */}

Thread Runtime State

SELECTED AGENT {activeAgentForThread?.name} {activeAgentForThread?.image}
MCP TOOL BINDINGS
{activeAgentForThread?.tools.map((t, i) => (
✓ {t}
)))}
ACTIVE MODEL PROVIDER {activeAgentForThread?.provider}
Zero-Code Container

This user session is completely isolated inside the OCI deployment.

)} {/* TAB 3: LOGS & TELEMETRY CONSOLE */} {activeTab === 'logs' && (
{/* Telemetry Visual Stat Cards */}
CPU WORKLOAD

{metrics.cpu}%

RAM ALLOCATED

{metrics.ram} GB

RUNNING CONTAINERS

{metrics.activeContainers}

MCP CONNECTIONS

{metrics.activeApiConns}

{/* Streaming Terminal Log Stream */}

Container Stdout Stream
LIVE LOG STREAM
{logs.map((log, index) => (
{index + 1} {log}
))}
)} {/* TAB 4: MCP GATEWAY SETTINGS */} {activeTab === 'settings' && (

Model Context Protocol (MCP) Hub Config

Configure internal standard MCP gateways that are injected automatically into customer container images upon deployment.

Zendesk Customer Portal MCP

http://zendesk-mcp:8080/mcp

Connected

PostgreSQL Docs Database MCP

http://pg-mcp:5432/mcp

Connected

Brave Web Search Gateway

http://brave-mcp:8000

Connected
)}
{/* --- MODAL 1: YAML CONFIG EDITOR MODAL --- */} {editingYamlAgent && (

Edit Config: {editingYamlAgent.name}

{editingYamlAgent.image}

setEditingYamlAgent(null)} className="text-slate-400 hover:text-slate-200 p-1 rounded-lg" >

setYamlContent(e.tar